

Network Simulation & Emulation Environment (NSE2) Manual

Lars Baumgaertner

2026-09-17 Work in Progress

Table of Contents

1. Introduction	1
2. Installation	2
2.1. Prerequisites	2
2.2. Installation Steps	2
3. Running NSE2	3
3.1. Example Scenario Overview	3
3.2. Starting a Simulation	6
3.3. Interacting with the Simulation using the Command Line Interface (CLI)	10
3.4. Interacting with the Simulation using the Management Web UI	10
3.5. Visualizing the Simulation	10
3.6. External WireGuard Service Example	12
3.6.1. Compose Topology	12
3.6.2. Gateway and iptables Configuration	13
3.6.3. Configuration	13
3.6.4. Starting and Verifying the Example	14
3.6.5. Running the NSE2 Tools	15
3.6.6. Stopping the Scenario	15
4. Creating a New Simulation Scenario	16
4.1. Using CCSDS Reference Scenarios and CSV Tools	16
4.1.1. Contact CSV	16
4.1.2. Generate a Contact Plan	17
4.1.3. Generate a Compose Topology	17
4.1.4. Generate Both Files	18
4.1.5. Shortening Realistic Contact Plans	18
4.2. Manually Creating a Scenario	18
4.2.1. Create a Docker Compose File	18
4.2.2. Optionally: Create a Contact Plan	19
4.2.3. Optionally: Create an Actions File	19
4.2.4. Optionally: Create a Visualization File	19
4.3. Using netedit to Create a Scenario (Work in Progress)	19
4.3.1. Using netedit	20
4.3.2. Converting GraphML to Docker Compose	21
5. File Formats	22
5.1. <i>Topology</i> File Format (compose.yml)	22
5.1.1. Setting up the Nodes	23
5.1.2. Setting up the Networks	24
5.2. <i>Core Contact Plan</i> File Format (contacts.ccp)	25
5.3. <i>Actions</i> File Format (actions.txt)	26

5.4. <i>Visualization</i> File Format (<i>viz.json</i>)	28
5.5. Optional: Node File Format (<i>node.yml</i>).....	30

Chapter 1. Introduction

The Network Simulation & Emulation Environment (NSE2) is a powerful tool designed to facilitate the simulation and emulation of network environments. This manual provides a comprehensive guide to installing, configuring, and using NSE2 effectively. It also includes instructions for creating new simulation scenarios and understanding the file formats used by NSE2.

NSE2 is built on top of Docker and Docker Compose, allowing users to create complex network topologies and simulate various network conditions. It is intended for researchers, developers, and network engineers who need to test and validate network protocols, applications, and configurations in a controlled realtime environment.

Chapter 2. Installation

NSE2 was designed to be easy to install and set up, allowing users to quickly get started with network simulation and emulation. While it has been designed mainly for Linux systems, it can in theory run on any system that supports Docker. This section provides detailed instructions on how to install NSE2 on your system.

2.1. Prerequisites

Before installing NSE2, ensure that you have the following prerequisites:

- A compatible operating system (Linux, macOS, or Windows)
- Python 3.11 or higher
- pip (Python package installer)
- Git (for cloning the repository)
- Docker (for containerized environments)
- Docker Compose



Podman is not supported. NSE2 relies on Docker-specific network interface naming; see [Section 5.1](#) for details.

2.2. Installation Steps

1. Clone the NSE2 repository from GitHub: `git clone https://github.com/esa/nse2.git`
2. Navigate to the NSE2 directory: `cd nse2`
3. Create and activate a virtual environment, then install the project and its dependencies:

```
python3 -m venv .venv
source .venv/bin/activate
pip install ".[dev]"
```

All `nse2_*` tools and helpers are installed into the virtual environment and available on PATH whenever the venv is active. `PYTHONPATH` is handled automatically.



For pinning dependencies to the exact versions tested in this repository, add `-c constraints.txt` to the `pip install` command.

4. **For development:** use an editable install so that changes to the source code take effect immediately without reinstalling:

```
pip install -e ".[dev]"
```

Chapter 3. Running NSE2

In this section, we will cover how to run NSE2, including starting a simulation as well as interacting with the simulation using the Command Line Interface (CLI) ([Section 3.3](#)), the Management Web UI ([Section 3.4](#)) and the visualization frontend ([Section 3.5](#)). We will also provide an example scenario to demonstrate how to set up and run a simulation in NSE2.

3.1. Example Scenario Overview

For demonstration purposes, we will use the `simple` scenario, located in `scenarios/simple/`, which is a basic network simulation scenario included with NSE2. Here, we have three nodes (n1, n2, and n3) connected in a straight line ([Figure 1](#)).

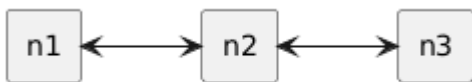


Figure 1. Network topology diagram of the `simple` scenario

In our example scenario we can find the following files:

```
$ ls scenarios/simple/
compose.yml  contacts.ccp  actions.txt  viz.json  run.sh
```

This is the minimal set of files required to run a simulation in NSE2 that has a topology, fluctuating contacts, and automated actions plus some optional information for the visualization frontend.

The `compose.yml` file defines the network topology and the nodes in the simulation, while the `contacts.ccp` file defines the contact plan for the nodes, the `actions.txt` file defines actions to be performed during the simulation, and the `viz.json` file provides visualization data.

The example `compose.yml` file ([Listing 1](#)) defines three nodes (n1, n2, and n3) connected in a straight line. Each node is configured with an IP address and a Docker image to run. The network interfaces are defined in the `networks` section, and the Docker Compose file specifies how the nodes are connected. A detailed description of the compose file format can be found in [Section 5.1](#).



The docker image must contain `tc` and `iproute2` tools to allow for network configuration and traffic control. In our example, we use the Alpine Linux image, which is lightweight and suitable for this purpose but may require additional packages to be installed.

Listing 1. Compose file of the `simple` scenario.

```
name: simple test scenario
services:
  n1:
    container_name: n1
    hostname: n1
```

```

cap_add:
- NET_ADMIN
privileged: 'true'
volumes:
- ./compose.yml:/compose.yml:ro
environment:
- NODE_ID=1
- TYPE=Host
networks:
  n1_n2:
    ipv4_address: 172.33.0.2
entrypoint: 'sh -c "apk add iproute2 bash && tail -f /dev/null"'
image: alpine
n2:
  container_name: n2
  hostname: n2
  cap_add:
  - NET_ADMIN
  privileged: 'true'
  volumes:
  - ./compose.yml:/compose.yml:ro
  environment:
  - NODE_ID=2
  - TYPE=Host
  networks:
    n1_n2:
      ipv4_address: 172.33.0.3
    n2_n3:
      ipv4_address: 172.33.1.2
  entrypoint: 'sh -c "apk add iproute2 bash && tail -f /dev/null"'
  image: alpine
n3:
  container_name: n3
  hostname: n3
  cap_add:
  - NET_ADMIN
  privileged: 'true'
  volumes:
  - ./compose.yml:/compose.yml:ro
  environment:
  - NODE_ID=3
  - TYPE=Host
  networks:
    n2_n3:
      ipv4_address: 172.33.1.3
  entrypoint: 'sh -c "apk add iproute2 bash && tail -f /dev/null"'
  image: alpine
networks:
  n1_n2:
    driver: bridge
    ipam:

```

```

    config:
      - subnet: 172.33.0.0/24
  driver_opts:
    com.docker.network.container_iface_prefix: n1_n2_
n2_n3:
  driver: bridge
  ipam:
    config:
      - subnet: 172.33.1.0/24
  driver_opts:
    com.docker.network.container_iface_prefix: n2_n3_

```

The contact plan ([Listing 2](#)) defines the communication links between the nodes, including bandwidth, delay, loss, and jitter. In our example, we have a fixed contact between n1 and n2 with a bandwidth of 100 Mbps and a delay of 100 ms, and a fluctuating contact between n2 and n3 with a bandwidth of 2 Mbps, a delay of 10 ms, and a time window from 20 to 40 seconds. Also, we set the contact plan to loop, meaning that the contacts will be repeated indefinitely. A detailed description of the contacts file format can be found in [Section 5.2](#).

Listing 2. Core contact plan of the simple scenario.

```

# Contact Plan
# Defines scheduled contacts and fixed links between nodes.
#
# Directives:
#   s loop <n>           : loop behaviour – 1 to repeat indefinitely, 0 (or omit) for no
loop
#   a <contact|fixed> : add a fixed link or a fluctuating contact with properties
described below
#
# Columns:
#   type      : 'a contact' for scheduled links, 'a fixed' for fixed links
#   start     : contact start time relative to scenario start (seconds, +offset)
#   end       : contact end time relative to scenario start (seconds, +offset)
#   src       : source node (node ID | node name)
#   dst       : destination node (node ID | node name | `dev:<interfacename>`)
#   bw        : bandwidth (e.g. 30mbit)
#   loss      : packet loss percentage (e.g. 0.0)
#   delay     : one-way propagation delay (ms)
#   jitter    : delay jitter (ms)
#   symmetric : '=' to apply the link in both directions, omit for one-way

s loop 1

# <type> <src> <dst> [bw] [loss] [delay] [jitter] [=]
a fixed  n1    n2   100mbit 0.0   100   0.0
a fixed  n2    n1   100mbit 0.0   100   0.0

# <type>  <start> <end> <src> <dst> [bw] [loss] [delay] [jitter] [=]
a contact 20     40    n2    n3   2mbit 0.0   10    0.0

```

```
a contact 20 40 n3 n2 2mbit 0.0 10 0.0
```

The actions file ([Listing 3](#)) defines the actions to be performed during the simulation. In our example, at first we log the start time on each node, then we schedule a repeated process on each node to log the link properties every 10 seconds by sending ping requests between the nodes. The actions file also includes a cleanup step that logs the finish time and terminates long-running processes at the end of the simulation. A detailed description of the actions file format can be found in [Section 5.3](#).

Listing 3. Example actions file of the simple scenario.

```
# DURATION is optional
# DURATION=300

# start a process to log the link properties every 10 seconds on the ground stations
EXEC 0 n1 "date > /tmp/started.txt"
EXEC 0 n2 "date > /tmp/started.txt"
EXEC 0 n3 "date > /tmp/started.txt"

# at timestamp 5 start to send ping (200 bytes) every 10s to gs1 from eosat
REPEAT 5 10 n1 "(date ; ping -c 1 -s 200 -W1 -w1 n2; echo "-----") >>
/tmp/n2.txt"

# at timestamp 10 start to send ping (200 bytes) every 10s to gs2 from eosat
REPEAT 10 10 n2 "(date ; ping -c 1 -s 200 -W1 -w1 n3; echo "-----")
>> /tmp/n3.txt"

# be nice and shut down long running processes
FINALLY n1 "date > /tmp/finished.txt && pkill -f ping"
FINALLY n2 "date > /tmp/finished.txt && pkill -f ping"
FINALLY n3 "date > /tmp/finished.txt && pkill -f ping"
```

3.2. Starting a Simulation

The following steps need to be performed to start a simulation in NSE2:

1. Start the topology: `nse2_topo compose.yml`
2. Start the contacts: `nse2_contacts -m compose.yml contacts.ccp`
3. Start the actions: `nse2_actions actions.txt`
4. Optionally, start the node manager: `nse2_mgr compose.yml contacts.ccp`

HINT: Use a terminal multiplexer like `tmux` or `screen` to run these commands in separate windows, allowing you to monitor the simulation in real-time.

Here, an example session starting the simulation.

```
$ nse2_topo compose.yml
```

Running scenario compose.yml

[+] Running 5/5

```
✓ Network simpletestscenario_n1_n2 Created 0.0s
✓ Network simpletestscenario_n2_n3 Created 0.0s
✓ Container n2 Started 0.4s
✓ Container n3 Started 0.4s
✓ Container n1 Started 0.4s
n1 | fetch https://dl-cdn.alpinelinux.org/alpine/v3.21/main/x86_64/APKINDEX.tar.gz
n2 | fetch https://dl-cdn.alpinelinux.org/alpine/v3.21/main/x86_64/APKINDEX.tar.gz
n3 | fetch https://dl-cdn.alpinelinux.org/alpine/v3.21/main/x86_64/APKINDEX.tar.gz
n2 | fetch https://dl-
cdn.alpinelinux.org/alpine/v3.21/community/x86_64/APKINDEX.tar.gz
n1 | fetch https://dl-
cdn.alpinelinux.org/alpine/v3.21/community/x86_64/APKINDEX.tar.gz
n3 | fetch https://dl-
cdn.alpinelinux.org/alpine/v3.21/community/x86_64/APKINDEX.tar.gz
n2 | (1/13) Installing ncurses-terminfo-base (6.5_p20241006-r3)
n2 | (2/13) Installing libncursesw (6.5_p20241006-r3)
n2 | (3/13) Installing readline (8.2.13-r0)
n2 | (4/13) Installing bash (5.2.37-r0)
n2 | Executing bash-5.2.37-r0.post-install
n2 | (5/13) Installing libcap2 (2.71-r0)
n2 | (6/13) Installing zstd-libs (1.5.6-r2)
n2 | (7/13) Installing libelf (0.191-r0)
n2 | (8/13) Installing libmnl (1.0.5-r2)
n2 | (9/13) Installing iproute2-minimal (6.11.0-r0)
n1 | (1/13) Installing ncurses-terminfo-base (6.5_p20241006-r3)
n2 | (10/13) Installing libxtables (1.8.11-r1)
n3 | (1/13) Installing ncurses-terminfo-base (6.5_p20241006-r3)
n2 | (11/13) Installing iproute2-tc (6.11.0-r0)
n1 | (2/13) Installing libncursesw (6.5_p20241006-r3)
n3 | (2/13) Installing libncursesw (6.5_p20241006-r3)
n2 | (12/13) Installing iproute2-ss (6.11.0-r0)
n1 | (3/13) Installing readline (8.2.13-r0)
n2 | (13/13) Installing iproute2 (6.11.0-r0)
n3 | (3/13) Installing readline (8.2.13-r0)
n2 | Executing iproute2-6.11.0-r0.post-install
n2 | Executing busybox-1.37.0-r12.trigger
n2 | OK: 11 MiB in 28 packages
n1 | (4/13) Installing bash (5.2.37-r0)
n3 | (4/13) Installing bash (5.2.37-r0)
n1 | Executing bash-5.2.37-r0.post-install
n1 | (5/13) Installing libcap2 (2.71-r0)
n3 | Executing bash-5.2.37-r0.post-install
n3 | (5/13) Installing libcap2 (2.71-r0)
n1 | (6/13) Installing zstd-libs (1.5.6-r2)
n3 | (6/13) Installing zstd-libs (1.5.6-r2)
n1 | (7/13) Installing libelf (0.191-r0)
n3 | (7/13) Installing libelf (0.191-r0)
n1 | (8/13) Installing libmnl (1.0.5-r2)
n3 | (8/13) Installing libmnl (1.0.5-r2)
```

```

n1 | (9/13) Installing iproute2-minimal (6.11.0-r0)
n3 | (9/13) Installing iproute2-minimal (6.11.0-r0)
n1 | (10/13) Installing libxtables (1.8.11-r1)
n3 | (10/13) Installing libxtables (1.8.11-r1)
n1 | (11/13) Installing iproute2-tc (6.11.0-r0)
n3 | (11/13) Installing iproute2-tc (6.11.0-r0)
n1 | (12/13) Installing iproute2-ss (6.11.0-r0)
n3 | (12/13) Installing iproute2-ss (6.11.0-r0)
n1 | (13/13) Installing iproute2 (6.11.0-r0)
n3 | (13/13) Installing iproute2 (6.11.0-r0)
n1 | Executing iproute2-6.11.0-r0.post-install
n1 | Executing busybox-1.37.0-r12.trigger
n1 | OK: 11 MiB in 28 packages
n3 | Executing iproute2-6.11.0-r0.post-install
n3 | Executing busybox-1.37.0-r12.trigger
n3 | OK: 11 MiB in 28 packages

```

Now all three nodes are running and ready to be configured.

Next step is to start the contact player, which will establish the communication links between the nodes based on the contact plan defined in `contacts.ccp`.

```

$ nse2_contacts -l -m compose.yml contacts.ccp
Loading scenario from compose.yml.
Node ipn:1.0 connected to network n1_n2 with 172.33.0.2
Node ipn:2.0 connected to network n1_n2 with 172.33.0.3
Node ipn:2.0 connected to network n2_n3 with 172.33.1.2
Node ipn:3.0 connected to network n2_n3 with 172.33.1.3
Created 3 nodes.
[INIT] Initialize contact: interface n1_n2_0 on node n2
[INIT] Initialize contact: interface n1_n2_0 on node n1
[INIT] Initialize contact: interface n2_n3_0 on node n2
[INIT] Initialize contact: interface n2_n3_0 on node n3
[ 0 ] Activating Contact (n1 -> n2 via n1_n2, 0--1)
[ 0 ] Activating Contact (n2 -> n1 via n1_n2, 0--1)
[ 0 ] Next event(s) at 20
[ 20 ] Activating Contact (n2 -> n3 via n2_n3, 20-40)
[ 20 ] Activating Contact (n3 -> n2 via n2_n3, 20-40)
[ 20 ] Next event(s) at 40

```

Now, we can start the actions defined in `actions.txt`, which will execute the actions during the simulation.

```

$ nse2_actions actions.txt
Warning: DURATION variable is not, REPEATING till infinity
0 5 10
[0] Executing commands on: n1 n2 n3
[0] n1: date > /tmp/started.txt
[0] n2: date > /tmp/started.txt

```

```

[0] n3: date > /tmp/started.txt
[5] Executing commands on: n1
[5] / 10 n1: (date ; ping -c 1 -s 200 -W1 -w1 n2; echo -----) >>
/tmp/n2.txt
[5] n1: (date ; ping -c 1 -s 200 -W1 -w1 n2; echo -----) >>
/tmp/n2.txt
[10] Executing commands on: n2
[10] / 10 n2: (date ; ping -c 1 -s 200 -W1 -w1 n3; echo -----) >>
/tmp/n3.txt
[10] n2: (date ; ping -c 1 -s 200 -W1 -w1 n3; echo -----) >>
/tmp/n3.txt
[15] n1: (date ; ping -c 1 -s 200 -W1 -w1 n2; echo -----) >>
/tmp/n2.txt
[20] n2: (date ; ping -c 1 -s 200 -W1 -w1 n3; echo -----) >>
/tmp/n3.txt
[25] n1: (date ; ping -c 1 -s 200 -W1 -w1 n2; echo -----) >>
/tmp/n2.txt
[30] n2: (date ; ping -c 1 -s 200 -W1 -w1 n3; echo -----) >>
/tmp/n3.txt
[35] n1: (date ; ping -c 1 -s 200 -W1 -w1 n2; echo -----) >>
/tmp/n2.txt
[40] n2: (date ; ping -c 1 -s 200 -W1 -w1 n3; echo -----) >>
/tmp/n3.txt
[45] n1: (date ; ping -c 1 -s 200 -W1 -w1 n2; echo -----) >>
/tmp/n2.txt
[50] n2: (date ; ping -c 1 -s 200 -W1 -w1 n3; echo -----) >>
/tmp/n3.txt
[55] n1: (date ; ping -c 1 -s 200 -W1 -w1 n2; echo -----) >>
/tmp/n2.txt
[60] n2: (date ; ping -c 1 -s 200 -W1 -w1 n3; echo -----) >>
/tmp/n3.txt
[65] n1: (date ; ping -c 1 -s 200 -W1 -w1 n2; echo -----) >>
/tmp/n2.txt
[70] n2: (date ; ping -c 1 -s 200 -W1 -w1 n3; echo -----) >>
/tmp/n3.txt
[75] n1: (date ; ping -c 1 -s 200 -W1 -w1 n2; echo -----) >>
/tmp/n2.txt
[80] n2: (date ; ping -c 1 -s 200 -W1 -w1 n3; echo -----) >>
/tmp/n3.txt
[85] n1: (date ; ping -c 1 -s 200 -W1 -w1 n2; echo -----) >>
/tmp/n2.txt
[90] n2: (date ; ping -c 1 -s 200 -W1 -w1 n3; echo -----) >>
/tmp/n3.txt
[95] n1: (date ; ping -c 1 -s 200 -W1 -w1 n2; echo -----) >>
/tmp/n2.txt
[100] n2: (date ; ping -c 1 -s 200 -W1 -w1 n3; echo -----) >>
/tmp/n3.txt
[105] n1: (date ; ping -c 1 -s 200 -W1 -w1 n2; echo -----) >>
/tmp/n2.txt
^CCleaning up...
n3: date > /tmp/finished.txt && pkill -f ping ; true

```

```
n2: date > /tmp/finished.txt && pkill -f ping ; true
n1: date > /tmp/finished.txt && pkill -f ping ; true
```

3.3. Interacting with the Simulation using the Command Line Interface (CLI)

Once the simulation is running, you can interact with it using the NSE2 Command Line Interface (CLI). With `nse2_sh` you can connect to any running node and start using it interactively.

```
$ nse2_sh n1
n1:/# ls /tmp/
n2.txt started.txt
n1:/# ping n2
PING n2 (172.33.0.3): 56 data bytes
64 bytes from 172.33.0.3: seq=0 ttl=64 time=0.271 ms
64 bytes from 172.33.0.3: seq=1 ttl=64 time=0.329 ms
^C
--- n2 ping statistics ---
2 packets transmitted, 2 packets received, 0% packet loss
round-trip min/avg/max = 0.271/0.300/0.329 ms
```



You can use all standard docker tools to interact with the nodes, such as `docker exec`, `docker logs`, and `docker ps`.

You can also interact with the simulation contacts backend using `nse2_cmd`. It can be used to get information about the current scenario, active links, simulation time and control the playback of the contacts file, e.g., pause/resume, skip to next event, etc.

3.4. Interacting with the Simulation using the Management Web UI

To control the simulation, you can use the Management Web UI (`nse2_mgr`), which provides a graphical interface to manage the simulation. Amongst others, it provides you with the following features: - View the current state of the simulation, including the nodes and contacts. - Access to shell and logs of each node. - Pause and skipping ahead for the contacts. - View and modification of dynamically managed contacts. - Activate and deactivate contacts. - Change bandwidth, delay, loss, and jitter of contacts.



Opening the logs or a terminal requires X11 forwarding (set `DISPLAY` environment variable) and an installed `xterm` on the host system running the manager.

3.5. Visualizing the Simulation

To visualize the simulation using `nse2_netviz`, you can use the `viz.json` file, which provides

visualization data for the nodes and contacts.

NSE2 provides a simple web-based visualization frontend to graphically represent the current state of the simulation. It displays a log file, the nodes, their (configured) positions, and the contacts between them. Optionally, it can also put a background image behind the nodes to provide a better context for the simulation, e.g., a map of the area where the nodes are located. The list of nodes also provides quick access to an interactive terminal for each node. Selecting a node opens a terminal connected to that node's container.

The visualization frontend is started by running the `nse2_netviz` command, which reads the `viz.json` file and starts a web server to serve the visualization.

```
nse2_netviz viz.json
```

```
{
  "title": "Simple NSE2 Example",
  "description": "A very simple three node dynamic network scenario.",
  "background": "extras/background.jpg",
  "links": "tmp/compose.netmap",
  "nodes": [
    {
      "name": "n1",
      "type": "computer",
      "x": 657,
      "y": 634,
      "color": "SkyBlue"
    },
    {
      "name": "n2",
      "type": "computer",
      "x": 693,
      "y": 384,
      "color": "SkyBlue"
    },
    {
      "name": "n3",
      "type": "settings_input_antenna",
      "x": 570,
      "y": 292,
      "color": "SkyBlue"
    }
  ]
}
```

A detailed description of the visualization file format can be found in [Section 5.4](#).



To get a proper visualization of the active links, `nse2_contacts` must be started with the `-m` option to periodically generate a mapping file with all active links.

The visualization frontend is very basic but can easily be extended to provide scenario and simulation specific information.

3.6. External WireGuard Service Example

The `wireguard` scenario demonstrates how an external application can participate in an NSE2 scenario through a WireGuard tunnel. It combines a Docker Compose topology on the NSE2 host with a second Compose project on the external host.

The NSE2 side contains two nodes, `n1` and `n2`, and a virtual node named `n3_wg`. The virtual node runs the WireGuard server and forwards traffic to a peer on the external host. The external host runs the WireGuard client and an HTTP application in the same network namespace.

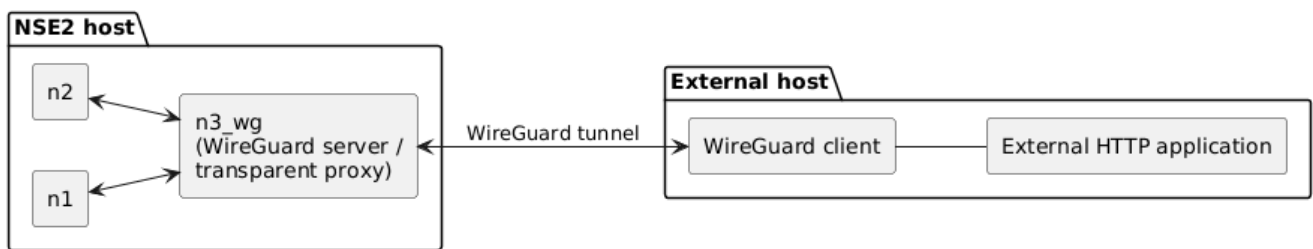


Figure 2. WireGuard external service topology



A WireGuard gateway node supports exactly one external WireGuard peer. Each virtual gateway node (`n3_wg`) is designed to represent a single external service reachable through a single WireGuard client. To connect multiple external services, deploy additional WireGuard gateway nodes, one per service.

The complete scenario is in `scenarios/wireguard/`. The generic Compose and contact plan formats are described in [Section 5.1](#) and [Section 5.2](#). This section focuses on the parts that differ from the standard container-only scenarios.

3.6.1. Compose Topology

The scenario uses two Compose files because the external application is not an NSE2 node. The important parts of `compose.yml` are:

- `n3_wg` is a specialized gateway service rather than a normal NSE2 node. It is a WireGuard server, but to NSE2 and the other nodes it appears as node `n3`.
- `n1` and `n2` are regular NSE2 nodes.
- `compose-client.yml` runs the external application with `network_mode: service:wg-client`. The application therefore shares the WireGuard client's network namespace and needs no separate route to use the tunnel.

The three networks in `compose.yml` have distinct purposes:

Network	Purpose
<code>n1_n3_wg</code> and <code>n2_n3_wg</code>	The simulated links between each NSE2 node and <code>n3_wg</code> . These are the links named by <code>contacts.ccp</code> and therefore receive the simulated bandwidth, delay, and availability conditions.
<code>wg_transport</code>	A high-priority gateway network used for communication with the host and the external WireGuard endpoint. It is deliberately not listed in the contact plan, so the outer WireGuard packets are not shaped as they leave the host.

This separation is important: the tunnel remains available to carry traffic, while the traffic path inside NSE2 can still be made intermittent or delayed. The generated peer configuration uses `ALLOWEDIPS` to route the WireGuard subnet and the NSE2 node networks through the tunnel.

3.6.2. Gateway and iptables Configuration

The server template installs `setup-iptables.sh` as both a `PostUp` and a `PostDown` hook for the `wg0` interface. The script is based on the container's actual interfaces and the generated peer configuration.

When the interface starts, the script:

- Finds every attached interface except loopback and `wg0`.
- Reads the single peer `/32` address and the WireGuard listen port with `wg show`.
- Allows forwarding between the peer and each Docker network and adds masquerading for return traffic.
- Accepts the outer WireGuard UDP port before the catch-all rule.
- DNATs other traffic addressed to a local `n3_wg` Docker address through the tunnel to the generated peer address, then permits the translated traffic through `wg0`.
- Stores the peer address and port in `/run` so that `PostDown` can remove the same rules after the WireGuard interface has been removed.

Rules are added only when absent and removed until absent, making repeated `up` and `down` calls safe. The script expects exactly one peer, which matches this example's one external service per WireGuard gateway.

3.6.3. Configuration

Before starting the NSE2 topology, review these values in `scenarios/wireguard/compose.yml`:

- `SERVERURL` is the address through which the external host reaches the NSE2 host. For a same-host test, `host.docker.internal` can be used.
- `SERVERPORT` must match the published UDP port for `n3_wg`.
- `ALLOWEDIPS` must contain the NSE2 networks that the external application should reach through the tunnel. Here, a list of all `/24` networks can be used or directly a bigger `/20` for example.

The example uses the [LinuxServer.io WireGuard image](#). Refer to its documentation for other container or host requirements.

3.6.4. Starting and Verifying the Example

Run the following command from `scenarios/wireguard/` on the NSE2 host:

```
nse2_topo compose.yml
```

The `n3_wg` container generates the server keys and the peer configuration under `server/runtime`, indicated by the line in the log:

```
n3_wg | **** No wg0.conf found (maybe an initial install), generating 1 server and  
external peer/client confs ****  
n3_wg | PEER external QR code (conf file is saved under /config/peer_external):
```

and bind mounted to the host at: `server/runtime/peer_external/peer_external.conf`.

Copy it to `client/wg0.conf` on the external host. For a same-host test:

```
cp server/runtime/peer_external/peer_external.conf client/wg0.conf  
chmod 600 client/wg0.conf
```

For a separate host, use a mechanism such as `scp` and apply the restrictive permissions there. The generated file contains the peer's private key and should not be shared.

Start the external WireGuard client and application from the directory that contains `compose-client.yml`:

```
docker compose -f compose-client.yml up
```

If the wireguard config was correctly copied the client should log:

```
external-wg-client | **** Found WG conf /config/wg_confs/wg0.conf, adding to list  
****
```

After that, check the handshake on both sides. Run the first command on the NSE2 host and the second command on the external host:

```
docker compose exec n3_wg wg show  
docker compose -f compose-client.yml exec wg-client wg show
```

It should look something like this:

```
interface: wg0
  public key: R/t7uUEJbPYZDxvAsLX9UNxGtxLR+zPnHEIEbhpAcQQ=
  private key: (hidden)
  listening port: 51820

peer: 1dRTEhLIJsX8uXaoT0d8g7vsuzw3/P4xLWgomm2Fhgs=
  preshared key: (hidden)
  endpoint: 172.32.0.1:51820
  allowed ips: 10.192.122.2/32
  latest handshake: Now
  transfer: 520 B received, 184 B sent
```

The **latest handshake** value should be a few seconds old. If there is no handshake, check **SERVERURL**, UDP reachability for **SERVERPORT**, and the copied peer configuration. Your WireGuard tunnel was not setup correctly.

The example application listens on port 80. Test traffic in both directions:

```
docker compose exec n1 curl --fail http://n3_wg
docker compose exec n2 curl --fail http://n3_wg
docker compose -f compose-client.yml exec wg-client ping -c 3 n1
docker compose -f compose-client.yml exec wg-client ping -c 3 n2
```

The first two requests should return **Hello from external WireGuard service!**.

3.6.5. Running the NSE2 Tools

After the tunnel is established, use the standard NSE2 tools described in [Chapter 3](#) and proceed as normal, starting the contact player with the scenario's topology and contact plan, setting up visualisation and starting the actions.

The action file provides a small ping and HTTP workload. The general syntax and behavior is covered by [Section 5.3](#).

3.6.6. Stopping the Scenario

Stop the action runner, contact player and topology as normal with **Ctrl+C**. Then stop the external Compose project on the external host:

```
docker compose -f compose-client.yml down
```

Remove **client/wg0.conf** and the generated contents of **server/runtime** when the test credentials are no longer needed.

Chapter 4. Creating a New Simulation Scenario

The recommended way to create an NSE2 scenario is to start with a scenario in the [CCSDS DTN Reference Scenarios](#) format. The repository provides the scenario data as CSV files; these can be converted directly into the NSE2 contact plan and Docker Compose files. The generated files keep node names and links synchronized, avoiding the need to understand the specifics of the [CCP format](#) and [Compose format](#).

The CSV scenario can be used unchanged, adapted before conversion, or used as the basis for a more substantial modification. Alternatively, an already converted scenario from NSE2's [scenarios/](#) directory can be copied and adapted. Creating both files manually should generally be avoided, but remains useful for small examples or special-purpose topologies. [netedit](#) is another alternative for graphical topology editing, though its GraphML conversion to NSE2 files is currently not yet supported.

4.1. Using CCSDS Reference Scenarios and CSV Tools

The following commands implement the CCSDS CSV workflow described above. Use the same contact CSV to generate both a contact plan and a Docker Compose topology.

The generators are available as the [csv_to_ccp](#) and [csv_to_compose](#) commands (or directly under [tools/helpers/](#) in a source checkout).

4.1.1. Contact CSV

The input CSV has a header followed by one row per directed link:

Source	Destination	Start	End	Bandwidth	Delay	Label
Node name	Node name	Seconds	Seconds	Bits per second	Milliseconds	Optional

The header row is skipped. The label column is optional. Loss and jitter are not CSV columns and are generated as zero. A row with [start=0](#) and [end=-1](#) becomes a fixed link. Matching rows in opposite directions are combined into a symmetric link where their timing and link properties match.

Use the [Label](#) column to distinguish multiple links between the same pair of nodes when they have different properties. Generated Docker network and interface names are based on the node names and, where needed, the link label. Keep node names short; if a generated name exceeds Linux's 15-character interface-name limit, it is replaced by a 12-character MD5 prefix. A long common prefix can sometimes be avoided with [--strip-prefix PREFIX](#); link labels ending in [high](#) or [low](#) are shortened to [hi](#) or [lo](#) when names are constructed.

4.1.2. Generate a Contact Plan

```
csv_to_ccp contacts.csv --output contacts.ccp
```

The available options are:

- `--output`, `-o`: output file; `-` writes to standard output.
- `--strip-prefix PREFIX`: remove `PREFIX` from source and destination names.
- `--speedup FACTOR`: divide contact times by `FACTOR` to shorten a scenario.
- `--bw-scale FACTOR` or `--bandwidth-scale FACTOR`: scale bandwidth values.

When multiple labelled links connect the same pair of nodes, the label is used to select a destination interface such as `dev:n1_n2_payload`.

4.1.3. Generate a Compose Topology

```
csv_to_compose contacts.csv --output compose.yml
```

The optional node mapping JSON supplies stable metadata for each CSV node. Its entries contain `node_label`, `node_name`, and `node_id`:

```
[
  {
    "node_label": "n1",
    "node_name": "spacecraft-1",
    "node_id": "ipn:1.0"
  }
]
```

If a mapping is not supplied, metadata is generated from the CSV labels. Use the mapping when the CCSDS scenario provides stable names and IDs.

The available `csv_to_compose` options are:

- `--output`, `-o`: Compose output file; `-` writes to standard output.
- `--strip-prefix PREFIX`: remove `PREFIX` from node labels.
- `--nodes FILE` or `--mapping FILE`: read node metadata from JSON.
- `--name NAME`, `-n`: set the Compose project name.
- `--image IMAGE`, `-i`: select the node image; the default is `alpine`.
- `--entrypoint COMMAND`, `-e`: override the default Alpine entrypoint.
- `--base-subnet PREFIX`: choose the first two subnet octets; the default is `172.33`.
- `--node-volumes DIRECTORY`: create and mount a data directory per node.

- `--build PATH, -b`: build nodes from the specified path instead of an image.
- `--mount-compose`: mount the generated Compose file read-only in each node.
- `--export-graphml FILE, -g`: additionally write the generated topology as GraphML. This is an output option; the script does not read GraphML as input.

The generator creates one Docker bridge network for each link, with predictable interfaces for the endpoints.

4.1.4. Generate Both Files

Generate the two NSE2 inputs from the same CCSDS contact CSV:

```
csv_to_ccp --strip-prefix eo actual_contacts.csv --output contacts.ccp
csv_to_compose --strip-prefix eo --nodes nodes.json \
  --name earth-orbit actual_contacts.csv --output compose.yml
```

Both generators record the generation date and command in their output. Review the generated files before starting the scenario, then use the normal NSE2 workflow from [Chapter 3](#).

4.1.5. Shortening Realistic Contact Plans

As an alternative to `csv_to_ccp`'s time and bandwidth scaling, `random_contacts` keeps the topology and link properties but replaces long, realistic contact windows with short random ones for a compact test run:

```
random_contacts contacts.ccp contacts-short.ccp \
  --length 300 --min-contact 5 --max-contact 20 --seed 42
```

The `--length` option sets the plan duration; `--min-contact`, `--max-contact`, and `--seed` control the generated windows. Fixed links and all link properties are preserved.

4.2. Manually Creating a Scenario

Manual creation is mainly useful for small examples or special-purpose topologies. The following sections describe the files that can make up an NSE2 scenario and how to create them using Docker Compose.

For a concrete example, refer to the simple scenario in the `scenarios/simple` directory. It contains three nodes connected in a line and is described in detail in [Chapter 3](#).

4.2.1. Create a Docker Compose File

Every new scenario starts with the creation of a Docker Compose file, which defines the network topology and the nodes in the simulation. The Compose file is written in YAML format and specifies the services (nodes), networks, and volumes that make up the simulation environment. It can be based upon an existing compose file, e.g., from a CI template or project repository, or you can start

from scratch. The details about the Compose file format can be found in [Section 5.1](#).

4.2.2. Optionally: Create a Contact Plan

The contact plan defines the communication links between the nodes and the link properties such as bandwidth, delay, loss, and jitter. It is written in a specific format that is parsed by the `nse2_contacts` tool during the simulation. The contact plan file is typically named `contacts.ccp` and is placed in the same directory as the Compose file. The details about the contact plan format can be found in [Section 5.2](#). Of course, if no traffic shaping and no dynamic links are needed, the contact plan can be empty or not used at all.

4.2.3. Optionally: Create an Actions File

The actions file defines the actions to be performed at specific points in time during the simulation, such as starting processes, sending messages, or logging information. It is written in a simple text format that is interpreted as a shell script and can include commands to be executed on the nodes. The actions file is typically named `actions.txt` and is placed in the same directory as the Compose file. The details about the actions file format can be found in [Section 5.3](#). If no actions are needed, the actions file can be empty or not used at all.

4.2.4. Optionally: Create a Visualization File

The visualization file is a JSON file that defines a scenario description, the nodes with their visualization properties, and where live link information can be found. It is used to visualize the network topology and the active links between the nodes. The visualization file is typically named `viz.json` and is placed in the same directory as the Compose file. The details about the visualization file format can be found in [Section 5.4](#). If no visualization is needed, the visualization file can be empty or not used at all.

4.3. Using `netedit` to Create a Scenario (Work in Progress)

To graphically create a simulation scenario, you can use the `netedit` tool. This tool provides a user-friendly interface for designing network topologies and configuring nodes and links. It is part of the [PONS-dtn](#) project, an open source discrete event simulator for delay-tolerant networks (DTNs), and allows one to export topology with link properties in [GraphML](#) format.

To use `netedit`, you must first install the `pons-dtn` package. You can do this by running the following command in your terminal:

```
pip install pons-dtn
```

You then can start `netedit` by running:

```
netedit
```



If you want to align positions of nodes with a background image you can point the **BG_IMG** environment variable to the image file you want to use as a background. For example:

```
BG_IMG=/path/to/your/background.png netedit
```

4.3.1. Using **netedit**

The UI provides a graphical representation of the network topology, allowing you to easily add, remove, and configure nodes and links.

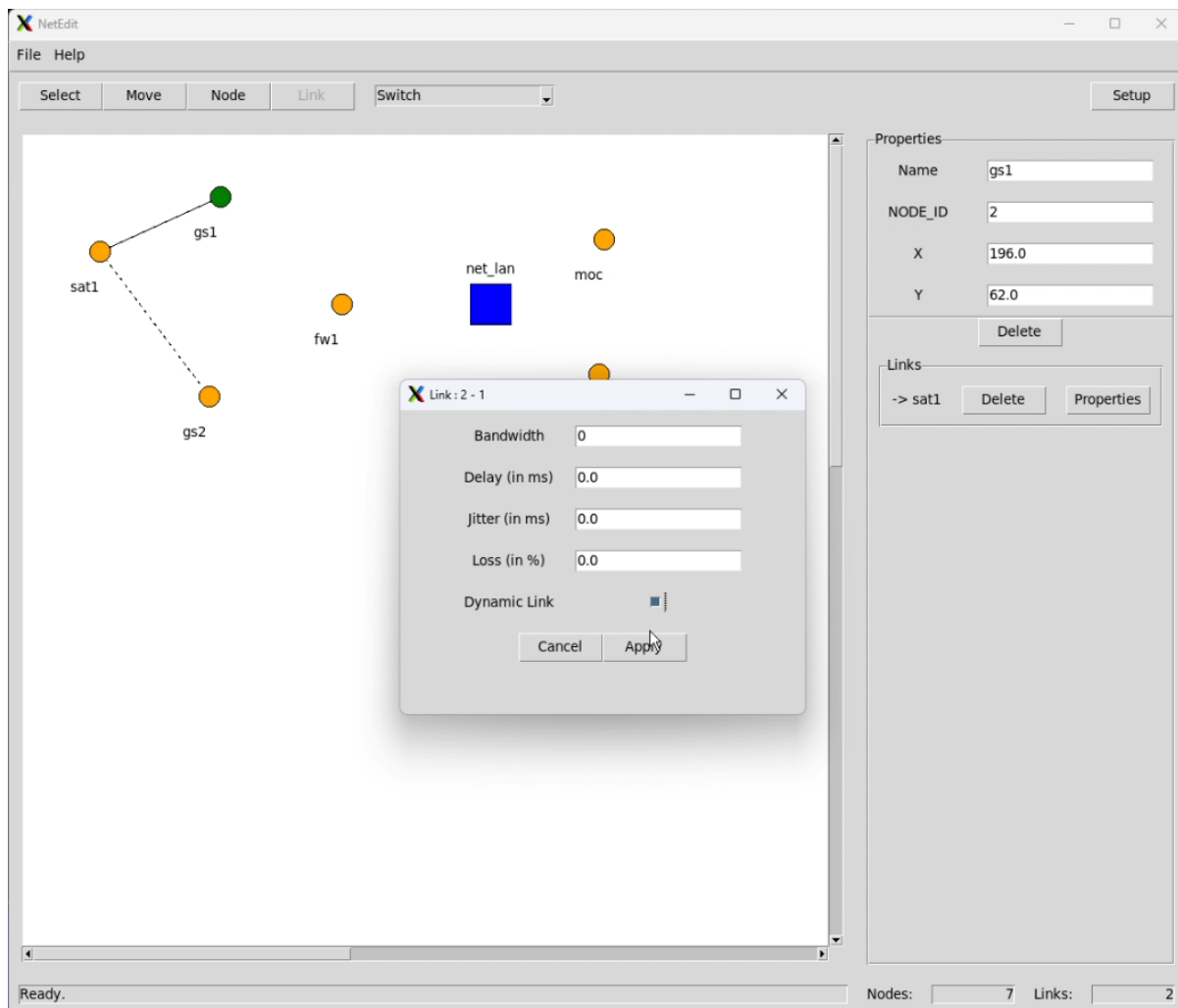


Figure 3. Netedit in action, editing the link properties of a connection between node 2 and 1.

There are 4 modes of operation as indicated by the buttons in the top left corner of the UI:

- **Select:** Select nodes to edit their properties.
- **Move:** To change the position of a node by dragging.
- **Node:** To add a new node to the topology, this can be either a *Switch* or a *Host*.
- **Link:** To create a new link between two nodes by dragging a line between them.

The currently selected node is always highlighted in green in the topology canvas.

Hosts can be either linked to another host via a 1:1 connection or to a switch, which can have multiple hosts connected to it. Switches are used to create a network segment where multiple hosts can communicate with each other.



Switch nodes should always have their names prefixed with "net_" so the automatic converters can treat them properly and it makes it easier to identify them later on.

On the right side of the UI, you can see the properties of the currently selected node and its links. You can edit the properties of the node, such as its name, type, and position, or the properties of the link, such as its bandwidth, delay, loss, and jitter.



The `NODE_ID` number must be unique in the whole topology!



In the link properties dialog, you can also set the link to be a dynamic link by checking the "Dynamic Link" checkbox. This will render the link as a dashed line, indicating that this is a non-permanent link.



Some advanced NSE2 features such as multiple parallel links between the same pair of nodes, or using specific network interfaces for the links, are not supported by `netedit` yet. You can still use the `compose.yml` file to define these features manually.

Via the *File* menu, you can save the current topology to a GraphML file.

4.3.2. Converting GraphML to Docker Compose

TODO

NSE2 currently has no helper scripts for converting GraphML to a Docker Compose topology or a contact plan.

Chapter 5. File Formats

5.1. *Topology* File Format (**compose.yml**)

The basic scenario and network topology is defined as a regular docker compose file.

You can start from scratch or use an existing compose file as a template. The compose file is written in YAML format and defines the services, networks, and volumes that make up the simulation environment.

```
name: simple test scenario
services:
  n1:
    container_name: n1
    hostname: n1
    cap_add:
      - NET_ADMIN
    privileged: 'true'
    volumes:
      - ./compose.yml:/compose.yml:ro
    environment:
      - NODE_ID=1
      - TYPE=Host
    networks:
      n1_n2:
        ipv4_address: 172.33.0.2
    entrypoint: 'sh -c "apk add iproute2 bash && tail -f /dev/null"'
    image: alpine
  n2:
    container_name: n2
    hostname: n2
    cap_add:
      - NET_ADMIN
    privileged: 'true'
    volumes:
      - ./compose.yml:/compose.yml:ro
    environment:
      - NODE_ID=2
      - TYPE=Host
    networks:
      n1_n2:
        ipv4_address: 172.33.0.3
      n2_n3:
        ipv4_address: 172.33.1.2
    entrypoint: 'sh -c "apk add iproute2 bash && tail -f /dev/null"'
    image: alpine
  n3:
    container_name: n3
    hostname: n3
```

```

cap_add:
- NET_ADMIN
privileged: 'true'
volumes:
- ./compose.yml:/compose.yml:ro
environment:
- NODE_ID=3
- TYPE=Host
networks:
  n2_n3:
    ipv4_address: 172.33.1.3
  entrypoint: 'sh -c "apk add iproute2 bash && tail -f /dev/null"'
  image: alpine
networks:
  n1_n2:
    driver: bridge
    ipam:
      config:
        - subnet: 172.33.0.0/24
    driver_opts:
      com.docker.network.container_iface_prefix: n1_n2_
  n2_n3:
    driver: bridge
    ipam:
      config:
        - subnet: 172.33.1.0/24
    driver_opts:
      com.docker.network.container_iface_prefix: n2_n3_

```

5.1.1. Setting up the Nodes

Each node is represented by a service in the compose file. In addition, one should explicitly set the `container_name` and `hostname` for each node to ensure that the names are consistent across different runs of the simulation.

Furthermore, for the network emulation to work correctly, the `NET_ADMIN` capability must be added to the container. This is done by setting the `cap_add` option in the compose file. And the container should run in `privileged` mode to allow full access to the network stack. Also, the ip address of the node for a specific network can be set using the `networks` option in the compose file.

You set the container `image` or alternatively the `build` option to specify the Dockerfile to build the image for the node.



The container images used for the nodes must have `iproute2` installed so that `tc` `qdisc` can be used to configure the network emulation for the link effects. Also, for very large delays, e.g., several minutes like to mars, a version of `iproute2` from [mid 2024](#) or later is required.

For scenarios managed by `nse2_contacts`, every node must define the `environment` variable `NODE_ID`

with a unique number. The contact player uses this value when resolving node references from the contact plan and applying its contacts. Applications can also use it for DTN auto-configuration.

The Compose and contact plan files can optionally be shared with a node as read-only mounts (`./compose.yml:/compose.yml:ro`, `./contacts.ccp:/contacts.ccp:ro`). This is useful when an application inside the node needs to inspect the topology or contact plan, for example to set up its own routing. The mounts are not required by the contact player itself.

Custom entryptoints and additional environment variables can be added when the node service needs application-specific configuration.



Sometimes it is useful to wrap your `entrypoint` commands in a `bash -c` command to ensure that the commands are executed in a shell environment. This is especially useful if you want to use shell features like variable expansion or command substitution. An example of this would be: `entrypoint: 'sh -c "apk add iproute2 bash && tail -f /dev/null"'`

5.1.2. Setting up the Networks

In general, it is advised to use a separate network for each pair of nodes. Thus, the network name should be unique for each pair of nodes, e.g., `n1n2` for the network between node 1 and node 2.



Network names must be unique across the entire compose file, so that they can be used to connect the nodes to the correct network. And, similar, to network interface names, network names must not contain any spaces or special characters, except for `-` and `_`, and best to keep them below 15 characters in length.

If no contact plan is used for a specific network segment, multiple nodes can be connected to the same network, e.g., `SWITCH1` for node 1 and node 2 and node 3. This is useful for scenarios where multiple nodes are connected to the same network segment, such as a LAN or a Wi-Fi network.



By default the `driver` should be set to `bridge`, which is the default Docker network driver. This allows the nodes to communicate with each other over the network.



With the default `bridge` driver, the network is not isolated from the host network, so that the nodes can access the host network (including the Internet) and vice versa. This can lead to unexpected behavior if the nodes are not properly configured to handle this. If you want to isolate the network from the host network, you can use the `macvlan` driver instead, which creates a virtual network interface for each node that is isolated from the host network.

Additionally, it can be helpful to set the `ipam` option to specify the IP address range for the network, e.g., `ipam: {config: [{subnet: "172.33.0.0/24"}]}`.

Finally, if you expect multiple links between the same pair of nodes, you can use specific network interfaces for the link (`com.docker.network.container_iface_prefix`). This prefix is used for the actual network interface name in the node container. Contact plans can then refer to these interfaces by their name, e.g., `dev:eth0` or `dev:n1_n2_0` for the second interface, and so on, instead of

using the node name as second node.



NSE2 relies on Docker's `com.docker.network.container_iface_prefix` option to assign predictable names to container network interfaces. Podman does not provide this Docker-specific behavior and is therefore not supported.



Network names in docker are global. Thus, running two scenarios with the same network names in parallel will lead to conflicts!

5.2. Core Contact Plan File Format (`contacts.ccp`)

The format of the contact plan file is `.ccp`, which stands for *Core Contact Plan* and is taken from the [Core Contact Manager](#), a fluctuating connectivity simulator for the [core network emulator](#). This file defines the communication links between nodes, including properties such as bandwidth, delay, loss, and jitter. The contact plan is essential for simulating realistic network conditions in NSE2.

Links between nodes can be fixed or fluctuating. Each link can be marked as symmetric by appending `=` as the final field, which causes `nse2_contacts` to apply the link properties in both directions. When `=` is omitted, the link is treated as one-way (asymmetric).

Each dynamic contact is defined by a time window, which specifies when the contact is active. Other tools can be used to simulate actual node movement and calculate visibility for the contact plan from this. The contact plan file is structured in a way that allows for easy parsing and integration with NSE2.

```
# Contact Plan
# Defines scheduled contacts and fixed links between nodes.
#
# Directives:
#   s loop <n>           : loop behaviour – 1 to repeat indefinitely, 0 (or omit) for no
loop
#   a <contact|fixed> : add a fixed link or a fluctuating contact with properties
described below
#
# Columns:
#   type      : 'a contact' for scheduled links, 'a fixed' for fixed links
#   start     : contact start time relative to scenario start (seconds, +offset)
#   end       : contact end time relative to scenario start (seconds, +offset)
#   src       : source node (node ID | node name)
#   dst       : destination node (node ID | node name | 'dev:<interfacename>')
#   bw        : bandwidth (e.g. 30mbit)
#   loss      : packet loss percentage (e.g. 0.0)
#   delay     : one-way propagation delay (ms)
#   jitter    : delay jitter (ms)
#   symmetric : '=' to apply the link in both directions, omit for one-way

s loop 1
```

```
# <type> <src> <dst> [bw] [loss] [delay] [jitter] [=]
a fixed n1 n2 100mbit 0.0 100 0.0
a fixed n2 n1 100mbit 0.0 100 0.0

# <type> <start> <end> <src> <dst> [bw] [loss] [delay] [jitter] [=]
a contact 20 40 n2 n3 2mbit 0.0 10 0.0
a contact 20 40 n3 n2 2mbit 0.0 10 0.0
```

Optionally, variables such as `loop` can be set to `1` to indicate that the contact plan should be repeated indefinitely if the simulation runs longer than the last timestamp. This can also be overridden by the `--loop` command line option when starting the simulation with `nse2_contacts`.

The first node must always be specified by its name, while the second node can be specified by its name or by its interface name (e.g., `dev:eth0`). This allows for flexibility in defining contacts, especially in scenarios where nodes have multiple interfaces to the same next-hop node or are part of a larger network topology.



For convenience, the bandwidth can be specified in various units, such as `kbit`, `mbit` or `gbit`.

```
a contact 20 40 n3 n2 2mbit 0.0 0.01 0.0
```



To mark a contact or fixed link as symmetric, append `=` as the last field on the line. This causes `nse2_contacts` to apply the link properties to both nodes automatically, without needing a second line for the reverse direction. The global `--symmetric` command-line option has been removed in favor of this per-contact approach.



You can interact with the contact plan file using `nse2_cmd` to get information about the running scenario, current links, and other properties. Additionally, you can control the playback of the contact plan, such as pausing or resuming the simulation, skipping to the next event, etc. This is useful for debugging and testing scenarios with dynamic contacts.

5.3. Actions File Format (`actions.txt`)

Action files define a number of actions or commands that are executed during different points in time during the simulation. These actions can be used to control the behavior of the nodes, such as starting or stopping services, sending messages, or changing configurations.

The actions file is written in a simple text format, where each line represents an action. A simple example can be seen in the following snippet:

```
# DURATION is optional
# DURATION=300

# start a process to log the link properties every 10 seconds on the ground stations
```

```

EXEC 0 n1 "date > /tmp/started.txt"
EXEC 0 n2 "date > /tmp/started.txt"
EXEC 0 n3 "date > /tmp/started.txt"

# at timestamp 5 start to send ping (200 bytes) every 10s to gs1 from eosat
REPEAT 5 10 n1 "(date ; ping -c 1 -s 200 -W1 -w1 n2; echo "-----") >>
/tmp/n2.txt"

# at timestamp 10 start to send ping (200 bytes) every 10s to gs2 from eosat
REPEAT 10 10 n2 "(date ; ping -c 1 -s 200 -W1 -w1 n3; echo "-----")
>> /tmp/n3.txt"

# be nice and shut down long running processes
FINALLY n1 "date > /tmp/finished.txt && pkill -f ping"
FINALLY n2 "date > /tmp/finished.txt && pkill -f ping"
FINALLY n3 "date > /tmp/finished.txt && pkill -f ping"

```



There is one important but *optional* global variable, **DURATION**, which is set to the duration of the simulation, or more precisely, the runtime of the action script in seconds. Without this the actions run until the action runner process is stopped.

There are three types of actions that can be defined in the actions file:

Table 1. Action Commands

Command	Parameters	Description
EXEC	<timestamp> <node_id> <command>	Executes a command <i>once</i> on the specified node at the given timestamp.
REPEAT	<timestamp> <interval> <node_id> <command>	<i>Repeats</i> a command on the specified node starting at the given timestamp with the specified interval.
FINALLY	<node_id> <command>	Executes a command on the specified node at the end of the simulation or when the action runner is stopped.



Action scripts are interpreted as shell scripts, thus, you can also use environment variables in your scripts. For example, you can set something like a shared **\$SEND_INTERVAL** to be reused by all **REPEAT** actions.



There can be only one entry per time and node or in the case of **FINALLY** one entry per node. If there are multiple entries for the same time and node, only the last one will be executed. This is to ensure that the actions are deterministic and do not interfere with each other. If you want to execute multiple commands at the same time, you can use a shell script that contains all the commands and call it from the action file or just chain them with **&&**.

5.4. Visualization File Format (**viz.json**)

The visualization file format is a JSON file that defines the nodes and links in the simulation scenario. It is used to visualize the network topology and the active links between the nodes. The file contains a list of nodes, each with a name, type, position (x and y coordinates), and color. Optionally, it can also put a background image behind the nodes to provide a better context for the simulation, e.g., a map of the area where the nodes are located as shown in [Figure 4](#).

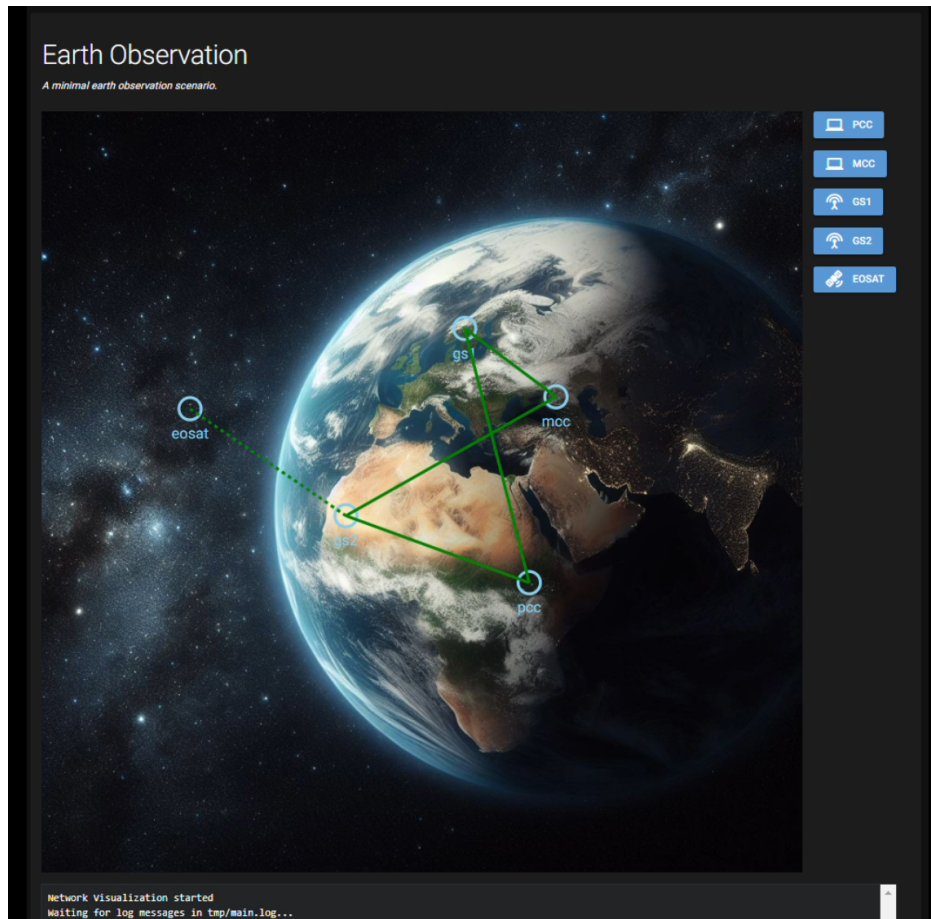


Figure 4. Example visualization file of an earth observation scenario.

The visualization frontend is started by running the `nse2_netviz` command, which reads the **viz.json** file and starts a web server to serve the visualization.

The **viz.json** file used for [Figure 4](#) is as follows:

```
{
  "title": "Earth Observation",
  "description": "A minimal earth observation scenario.",
  "background": "extras/background.jpg",
  "links": "tmp/compose.netmap",
  "nodes": [
    {
      "name": "pcc",
      "type": "computer",
      "x": 657,
      "y": 634,
```

```

    "color": "SkyBlue"
  },
  {
    "name": "mcc",
    "type": "computer",
    "x": 693,
    "y": 384,
    "color": "SkyBlue"
  },
  {
    "name": "gs1",
    "type": "settings_input_antenna",
    "x": 570,
    "y": 292,
    "color": "SkyBlue"
  },
  {
    "name": "gs2",
    "type": "settings_input_antenna",
    "x": 410,
    "y": 544,
    "color": "SkyBlue"
  },
  {
    "name": "sat",
    "type": "satellite_alt",
    "x": 200,
    "y": 400,
    "color": "SkyBlue"
  }
]
}

```

The most important top level keys in the JSON file are:

Table 2. Important visualization JSON keys

Key	Description
<code>title</code>	An optional title for the visualization.
<code>description</code>	An optional description of the visualization.
<code>background</code>	An optional background image to be displayed behind the nodes.
<code>links</code>	A file containing the current state of links.
<code>nodes</code>	A list of nodes in the simulation, each with a unique name, type, position, and color.

The `links` key is used to specify the file that contains the current state of links in the simulation. This file is generated by the `nse2_contacts` tool, which collects the active links between the nodes during the simulation. The file just contains a list of links, each pair on a separate line and a "-" or

"." indicating whether it is a fixed or dynamic link, e.g., $n1 - n2$ or $n1 . n2$.



To get a proper visualization of the active links, `nse2_contacts` must be started with the `-m` option to periodically generate a mapping file with all active links.

5.5. Optional: Node File Format (`node.yml`)

TODO

See CCSDS DTN Reference Scenarios Orange Book Draft for details on the node file format.